PROJECT THESIS REPORT

# ResolveDesk AI

An AI Customer-Support SaaS Product: Knowledge-Base Chat with Confidence-Gated Ticketing, Machine-Learning Triage and Deflection Analytics — A Product Thesis and Complete Implementation Report

HEADLINE RESULTS

**100%**
Intent accuracy
macro-F1 1.000, held-out test

**98.3%**
Priority accuracy
macro-F1 0.983

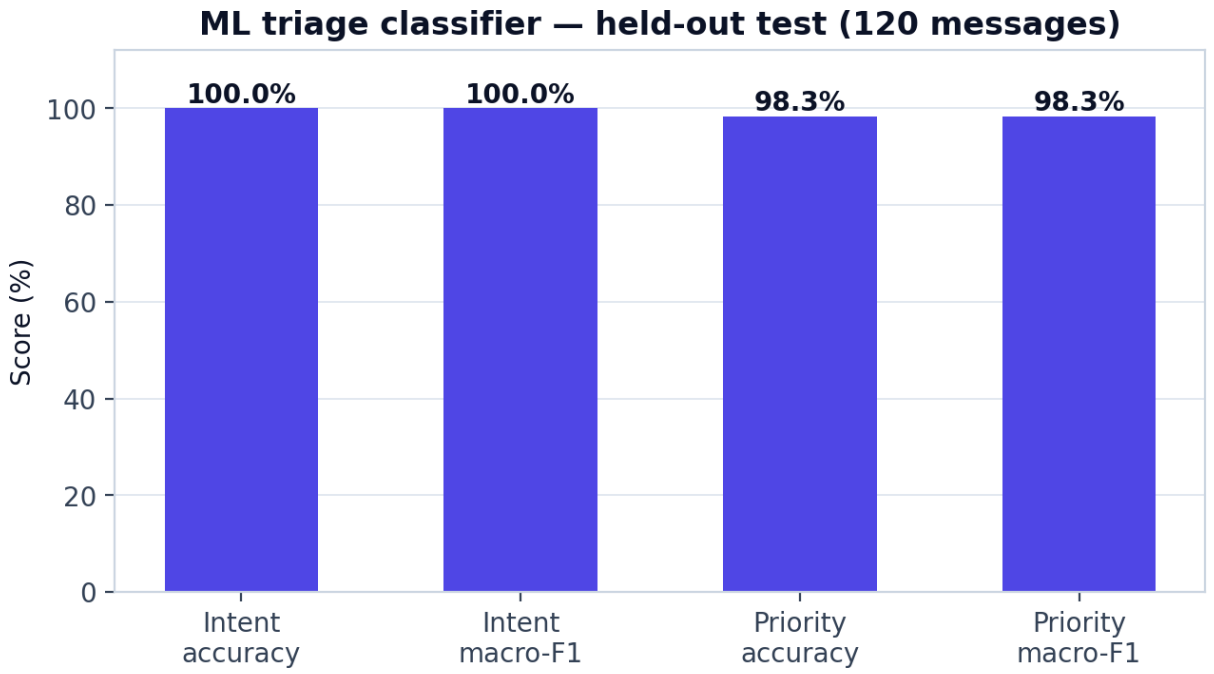
**13/13**
Product tests passing
incl. tenant isolation

**10/10**
Live HTTP smoke steps
full user journey

**Ashutosh Sahoo**
M.Tech, Artificial Intelligence · NIT Silchar
November 2026

# Results at a Glance

| | |
|---|---|
| **100%**<br>Intent accuracy<br>macro-F1 1.000, held-out test | **98.3%**<br>Priority accuracy<br>macro-F1 0.983 |
| **13/13**<br>Product tests passing<br>incl. tenant isolation | **10/10**<br>Live HTTP smoke steps<br>full user journey |




*Triage classifier performance on the 120-message held-out test set (600 generated messages, 480 train / 120 test). Synthetic-data scores are an upper bound — see the report's limitations chapter.*

# Contents

**Abstract**

**Chapter 1 — Introduction**

**Chapter 2 — Background and Theory**

**Chapter 3 — System Design and Architecture**

**Chapter 4 — Implementation: Every Feature and Every Function**

**Chapter 5 — Data: The Triage Corpus and the Seed Knowledge Bases**

**Chapter 6 — Experiments and Product QA Protocol**

**Chapter 7 — Results and Analysis**

**Chapter 8 — The Business: Market, Pricing, Unit Economics, Delivery**

**Chapter 9 — Discussion: Limitations, Threats to Validity, Production Path**

**Chapter 10 — Conclusion and Future Work**

**Chapter 11 — Learning Guide**

**Chapter 12 — How to Run**

**Chapter 13 — References**

**Appendix A — Full `results.json`**

**Appendix B — File Listing**

**Appendix C — Complete API Reference**

## Abstract

Small and medium businesses answer the same customer questions every day — where is my order, how long does a refund take, what are your timings — yet enterprise support platforms are priced per seat and per resolution in US dollars and expect the customer to configure everything themselves. This report presents **ResolveDesk AI**, a working multi-tenant AI customer-support product built end-to-end by the author: a small business registers, loads its own help articles into an isolated knowledge base (KB), embeds a chat widget on its website, and receives cited answers for its customers within seconds. When the system cannot answer confidently, it deliberately does **not** guess: it creates a support ticket instead, classifies that ticket with a machine-learning triage model (intent, priority, sentiment), and surfaces it — urgent first — in an owner dashboard that also reports how many conversations the AI resolved without human help (the *deflection rate*).

The product is implemented with deliberately minimal infrastructure — Python's standard-library `http.server` and `sqlite3`, plus NumPy for the machine learning — so that it runs anywhere with zero external APIs, zero API keys and zero per-conversation cost. The triage classifier (TF-IDF features feeding a from-scratch multinomial logistic regression with two heads, plus a lexicon sentiment analyser and a hybrid rule layer that always escalates explicit urgency signals) is trained on 600 generated support messages and evaluated on a stratified held-out set of 120: intent accuracy 100% (macro-F1 1.000), priority accuracy 98.3% (macro-F1 0.983), trained in 14.12 seconds on CPU. These numbers are reported with an explicit caveat, repeated throughout this report: the dataset is synthetic and template-generated, so the scores are an upper bound; in client delivery the classifier is re-tuned on each client's real ticket history. Product quality is evidenced not only by classifier metrics but by a 13-case assert-based test suite (including tenant-isolation and owner-only mutation tests) and a 10-step smoke test that performs the complete user journey over real HTTP — both fully passing. The report explains every feature and every function of the codebase, the theory underneath it, the experiments and their analysis, and — because this is a product, not only a programme — the business built around it: target market, pricing (₹4,999–₹19,999/month plus a ₹14,999 setup fee), clearly-labelled estimated unit economics, a 48-hour client onboarding playbook, honest competitor positioning against Intercom Fin and Zendesk AI, and a production roadmap. A learner should be able to rebuild, explain and sell this product from this document alone.


**Keywords:** SaaS, multi-tenancy, customer support automation, RAG-style retrieval, TF-IDF, cosine similarity, logistic regression, text classification, ticket triage, deflection rate, confidence gating, PBKDF2, product engineering, freelance services

# Chapter 1 — Introduction

## 1.1 The problem

Customer support for a small business has a peculiar shape. The *volume* is dominated by a small set of repeated questions: delivery status, refund timelines, prices, timings, return rules. Published industry experience and the author's own project research agree that a large share of tickets are near-duplicates of questions the business has already answered in its FAQs and policy pages. Yet the *cost* of answering them is human time, paid every single time, including at 2 a.m. when no human is available.

The existing solutions fit this shape badly:

**Enterprise AI support platforms** (Intercom Fin, Zendesk AI and similar) are powerful but priced per seat and per resolution in USD, and are self-serve: the business must configure, train and maintain the system itself. For an owner-run business with no support team, both the price curve and the configuration burden are wrong.

**Generic chatbots / LLM chat widgets** answer fluently whether or not they know the answer. In a support context, a confidently wrong refund promise is worse than no answer: it creates a commitment the business never made, and a furious customer when it is not honoured.

**A human-only inbox** (email/WhatsApp) gives no triage: an urgent "my account is compromised" waits in the same queue as "what are your working hours?", in arrival order.

## 1.2 The product idea

ResolveDesk AI is designed around one principle and one business model.

**The principle — answer only when you know, and prove it.** The chat engine may answer a customer only when it retrieves a help article from *that business's own knowledge base* with similarity at or above a fixed confidence threshold (0.12 TF-IDF cosine in this MVP). Every answer carries a **citation** — the title of the article it came from. Below the threshold, the system refuses to improvise: it creates a ticket, tells the customer plainly that it has done so, and hands the conversation to the human workflow, pre-classified by intent, priority and sentiment, sorted so urgent tickets are seen first.

**The business model — a done-for-you service plus subscription.** The software is sold not as raw software but as an installed, tuned, maintained service: the author sets it up on the client's website within 48 hours, loads the client's FAQs, tunes triage on the client's real tickets, trains the owner, and charges a setup fee plus a flat monthly subscription (Chapter 8 gives the full pricing and playbook). This matches how the target customers — D2C/e-commerce stores, local service businesses, small SaaS/ed-tech teams with roughly 200–5,000 support conversations a month and no dedicated support team — actually buy.

## 1.3 Objectives

1. **O1 — A real multi-tenant product.** Multiple independent businesses on one deployment, each with isolated KB, tickets, conversations, analytics and its own embeddable widget key. Isolation must be *tested*, not assumed.
2. **O2 — Grounded, cited answers with an honest refusal path.** Retrieval from the tenant's own articles only; citation on every answer; a ticket — never a guess — below the confidence

threshold.

3. **O3 — Machine-learning triage on every conversation.** Intent (6 classes), priority (4 classes) and sentiment, with a hybrid rule layer for explicit urgency, attached to every ticket and conversation record.
4. **O4 — Owner visibility.** A dashboard showing conversations, AI-resolved (deflection) rate, tickets, average confidence and intent breakdown, computed from real database rows — no decorative statistics.
5. **O5 — Zero-dependency deployability.** Standard library + NumPy only; one command to train, one to serve; automated tests and an HTTP smoke test that a buyer or interviewer can run.
6. **O6 — A sellable service pack.** Pricing, onboarding playbook, sales pitch, competitor positioning and roadmap, documented alongside the code.

## 1.4 Contributions

A complete, working SaaS MVP (~18 KB of server code, ~12 KB of ML code, three web pages) in which **every advertised feature is implemented and covered by an automated test** — multi-tenancy, KB management, cited retrieval, confidence-gated ticketing, ML triage, ticket workflow, analytics, owner authentication.

A from-scratch (NumPy-only) TF-IDF + multinomial logistic-regression triage classifier with two classification heads, lexicon sentiment and a production-style hybrid rule override, with a fully documented training/evaluation protocol and honestly-caveated results.

A QA story unusual for portfolio projects: 13 functional/isolation tests plus a 10-step real-HTTP user-journey smoke test, both passing, specified test-by-test in Chapter 6.

A business chapter that treats pricing, unit economics (explicitly labelled estimates), delivery and competition as engineering-grade artefacts of the product.

## 1.5 How to read this report

Chapters 2–3 give theory and design. Chapter 4 is the heart of the "learn from it" promise: every file, feature and function, in execution order, with design decisions and code excerpts. Chapters 5–7 cover data, experiments/QA and results with analysis. Chapter 8 is the business. Chapters 9–10 discuss limitations and conclude. Chapter 11 is a self-study and interview/client preparation guide.

# Chapter 2 — Background and Theory

## 2.1 Software as a service and multi-tenancy

A SaaS product serves many customers (*tenants*) from one running system. The central design decision is the **isolation model**: shared database with a tenant discriminator column, schema-per-tenant, or database-per-tenant. ResolveDesk uses the first — every tenant-owned table (`kb`, `conversations`, `tickets`, `sessions`) carries a `tenant_id`, and every query is scoped by it. This is the cheapest model to operate and the easiest to get wrong: a single unscoped query leaks one business's data to another. The defence used here is discipline plus verification: (i) tenant identity is derived from the session token or widget key on the server, never accepted from the request body; (ii) mutations include the tenant in the `WHERE` clause (`DELETE FROM kb WHERE id=? AND tenant_id=?`; ticket status updates likewise), so a guessed ID belonging to another tenant matches zero rows; (iii) isolation is an explicit test case (Chapter 6).

Two distinct credentials exist because the product has two distinct principals:

> The **owner session token** (issued at login, sent as `Authorization: Bearer …`) authorises management operations: KB edits, ticket lists and status changes, analytics.
>
> The **widget key** (`wk_` + 20 hex characters) is a public-by-design identifier embedded in the client's website that authorises exactly one operation — customer chat for that tenant. It is a *routing* credential, not an owner credential: possessing it lets you talk to that business's bot, nothing more. Keeping these two credential classes separate is what makes an embeddable widget safe to ship in page source.

## 2.2 Retrieval over a knowledge base: TF-IDF and cosine similarity

The chat engine is a retrieval system in the retrieval-augmented generation (RAG) family, minus the generative model: instead of synthesising an answer, it returns the best-matching article verbatim, with a citation. The ranking function is TF-IDF cosine similarity, the classical baseline of information retrieval (Salton's vector space model).

For a term $t$ in a document $d$ from a corpus of $N$ documents, the implementation in `app.py: retrieve()` uses a log-scaled term frequency and a smoothed inverse document frequency:

$$\text{tf-idf}(t, d) = (1 + \ln \text{count}(t,d)) \cdot \left( \ln((1+N)/(1+\text{df}(t))) + 1 \right)$$

where `df(t)` is the number of documents containing $t$. Log-scaling the count prevents one repeated word from dominating; the `+1` smoothing keeps every term's weight positive and defined for unseen terms. Query and document become sparse vectors in term space, and relevance is the cosine of the angle between them:

$$\cos(q, d) = (q \cdot d) / ( ||q|| \cdot ||d|| )$$

Cosine is length-invariant, which matters here: a two-line article and a ten-line article should compete on *topical overlap*, not raw word mass. Two product-specific refinements sit on top of the textbook formula (both in `_rtok()`): **stopword removal** (a 40-word list — "how", "what", "do", … — that carries no topical signal) and **light stemming** (stripping a trailing plural "s" on words longer than 3 characters, so "refunds" matches "refund"). Finally, article titles are weighted by duplication — the corpus text for each article is `title + title + body` — a simple, effective form of field weighting, since a query matching the title is usually exactly the article wanted.

## 2.3 Confidence gating and the deflection rate

Retrieval scores are only useful if the product *acts* on them. ResolveDesk sets an answer threshold, `CHAT_THRESHOLD = 0.12`: at or above it, answer with citation; below it, abstain and create a ticket. This is the support-domain instance of **selective prediction** (classification with a reject option): a system that may say "I don't know" can be far more valuable than one that must always answer, because the cost of the two error types is asymmetric — a missed answer costs a ticket (the normal human path anyway), while a fabricated answer costs trust and possibly money.

The headline product metric follows directly. If `resolved_by_ai` is recorded on every conversation (1 when answered from the KB, 0 when ticketed), then

```
deflection rate = (number of conversations with resolved_by_ai = 1) / (total conversations)
```

— the fraction of support load the AI absorbed. `analytics()` computes this, the raw counts and the mean retrieval confidence directly from the `conversations` table, together with a per-intent breakdown (`GROUP BY intent`), so every number on the dashboard is an auditable aggregate of real interactions.

## 2.4 Text classification: multinomial logistic regression

Triage is supervised multi-class classification, twice over: an **intent head** (billing, technical, delivery, refund, account, general) and a **priority head** (low, normal, high, urgent). Features are TF-IDF vectors (Chapter 5); the model is multinomial logistic regression (softmax regression), implemented from scratch in NumPy (`ml.py: SoftmaxLR`).

Given features $x$, weights $W$ and bias $b$, class probabilities are the softmax of the logits:

$$z = xW + b \qquad p(y=k \mid x) = \exp(z_k) / \Sigma_j \exp(z_j)$$

Training minimises cross-entropy loss with an L2 penalty by full-batch gradient descent. With one-hot targets $Y$ and predictions $P$, the gradient has the clean closed form used in the code:

$$G = (P - Y) / n \qquad W \leftarrow W - lr \cdot (X^{\top} G + \lambda W) \qquad b \leftarrow b - lr \cdot \Sigma\, G$$

The implementation subtracts the per-row maximum logit before exponentiating (the standard numerical-stability trick), uses `lr = 0.5`, `λ = 1e-4`, 300 epochs, and small Gaussian initialisation (`σ = 0.01`). Softmax regression is the right-sized model for this problem: the classes are separated largely by vocabulary, the model trains in seconds on CPU, its probabilities double as confidences, and — pedagogically important for this report — every line of it can be explained, unlike a downloaded black box.

**Evaluation metric — macro-F1.** Accuracy alone hides per-class failure (a model that never predicts "urgent" can still score well if urgent is rare). The report therefore uses macro-F1 alongside accuracy: for each class, precision `P = TP/(TP+FP)` and recall `R = TP/(TP+FN)` give `F1 = 2PR/(P+R)`, and macro-F1 is the unweighted mean over classes — every class counts equally, including the rare, important ones. `ml.py: macro_f1()` implements exactly this.

## 2.5 Lexicon sentiment and hybrid rule+ML systems

Sentiment is computed by the simplest defensible method: a **lexicon**. Two word sets (`POS_WORDS`, 16 words; `NEG_WORDS`, 21 words — note it deliberately includes support-domain negatives such as "refund", "broken", "delayed", "fraud") are counted over the message tokens; more negatives than positives  negative, more positives  positive, otherwise neutral. Lexicons are brittle (no negation, no sarcasm) but transparent, free, and adequate for a dashboard signal whose job is triage colour, not psychology.

The priority path demonstrates a pattern used throughout production ML: the **hybrid rule layer**. The ML head's prediction is overridden to `urgent` whenever the message contains an explicit urgency signal (`urgent`, `urgently`, `emergency`, `asap`, `immediately`, `critical`), and the response records `priority_source: "rule_override"` versus `"ml"` so the override is auditable, never silent. The rationale is cost asymmetry again: the downside of escalating a false alarm is a human glancing at a ticket early; the downside of an ML miss on "my account is compromised, ASAP" is a breach left in a queue. Production triage systems routinely hard-escalate such signals; the model handles the grey zone, the rule guards the cliff edge.

## 2.6 Password storage: salted PBKDF2

Owner passwords are never stored. `register_tenant()` generates a per-user salt (`secrets.token_hex(8)`) and stores `PBKDF2-HMAC-SHA256(password, salt, 60,000 iterations)` as hex (`app.py: _hash()`). Login recomputes the hash and compares. PBKDF2's iteration count makes brute force expensive; the per-user salt defeats rainbow tables and ensures identical passwords hash differently. Session tokens are 128-bit random values (`secrets.token_hex(16)`) stored server-side. Chapter 9 is explicit about what this MVP does *not* yet do — token expiry, rate limiting, HTTPS — and why those are roadmap items rather than hidden flaws.

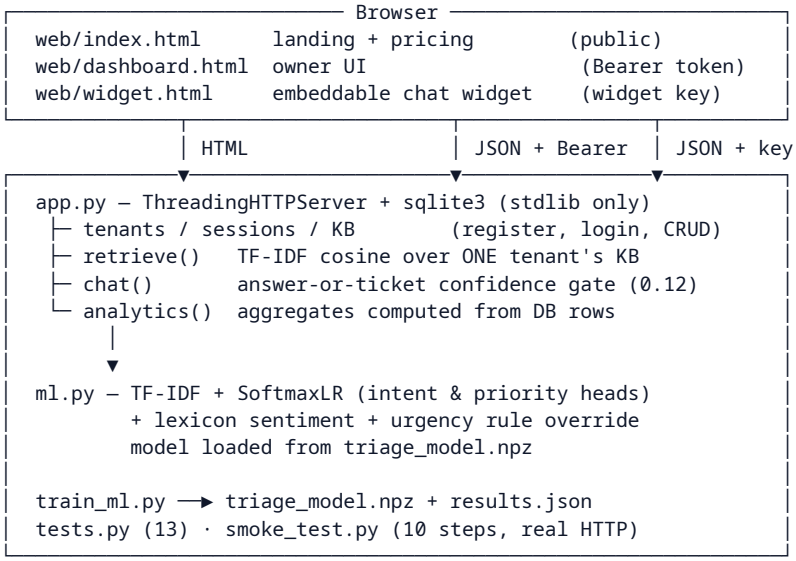
## 2.7 Where this sits in the state of the art

Contemporary support AI (Intercom Fin, Zendesk AI, and LLM-backed agents generally) replaces lexical retrieval with embedding retrieval and extractive answers with LLM generation, and adds tool use (order lookup, refunds) on top. ResolveDesk's architecture is deliberately shaped so that upgrade is a *component swap, not a redesign*: the retrieval function, the citation requirement and the confidence gate are exactly the seams at which an embedding retriever and an LLM answer layer attach (BUSINESS.md roadmap item 1 keeps the lexical engine as fallback). What the MVP optimises for instead is the part of the problem the giants under-serve: zero marginal cost, zero configuration burden for the owner, and a refusal path that makes a small model safe to put in front of real customers.

---

# Chapter 3 — System Design and Architecture

## 3.1 Architecture overview




There is no framework, no build step and no external service. The server is Python's `ThreadingHTTPServer`; persistence is a single SQLite file (`resolvedesk.db`, overridable via the `RESOLVEDESK_DB` environment variable — the mechanism both test suites use to run against throwaway databases); the ML model is a single compressed NumPy archive (`triage_model.npz`, 25 KB). The whole product can be copied to a small VPS and run with `python3 app.py`.

## 3.2 The two request flows that define the product

**Customer chat flow (`POST /api/chat`).** (1) The widget sends `{widget_key, message}`. (2) The server resolves the key to a tenant — an unknown key is a 404, nothing else executes. (3) The triage model classifies the message (intent, priority, sentiment, confidences, priority source). (4) `retrieve()` ranks *that tenant's* articles and returns the best article and its cosine score. (5) The gate: score  0.12 answer with the article body and its title as citation, log the conversation as AI-resolved; score < 0.12 (or empty KB)  insert the conversation as unresolved, insert a linked ticket carrying the triage, and return the "I've created a support ticket" message with the new `ticket_id`. Either way, exactly one conversation row is written — analytics can therefore never drift from reality.

**Owner flow.** Registration creates the tenant and its widget key. Login verifies the PBKDF2 hash and mints a session token. Every subsequent owner call (`/api/me`, `/api/kb`, `/api/tickets`, `/api/analytics`, ticket status) resolves the `Bearer` token to a tenant server-side and scopes all reads and writes to that tenant. Ticket status changes validate the requested value against the allowed set (`open`, `in-progress`, `resolved`) and the tenant scope in the same SQL statement.

## 3.3 Database schema

Five tables, created idempotently by `init_db()`:

| Table | Columns (key fields) | Purpose |
|---|---|---|
| `tenants` | id, name, industry, email UNIQUE, password_hash, salt, widget_key UNIQUE, created | One row per business; holds both credentials |

| Table | Columns (key fields) | Purpose |
|---|---|---|
| `sessions` | token PK, tenant_id | Bearer token  tenant mapping |
| `kb` | id, tenant_id, title, body | Help articles, tenant-scoped |
| `conversations` | id, tenant_id, message, answer, citation, confidence, intent, priority, sentiment, resolved_by_ai, created | Append-only log of every chat; the analytics source of truth |
| `tickets` | id, tenant_id, conversation_id, message, intent, priority, sentiment, status (default `open`), created | Escalations, linked to their conversation |

## 3.4 Module map

| File | Role |
|---|---|
| `app.py` | Server, auth, tenancy, KB, retrieval, chat gate, tickets, analytics, demo seeding, HTTP routing |
| `ml.py` | Dataset generation, tokeniser, TF-IDF, softmax logistic regression, sentiment, macro-F1, `TriageModel` (train/evaluate/predict/save/load) |
| `train_ml.py` | Stratified split, training run, metrics, model + `results.json` persistence |
| `tests.py` | 13 assert-based functional/isolation tests against a temp DB |
| `smoke_test.py` | 10-step end-to-end journey over real HTTP against a self-started server |
| `web/index.html` | Landing page + pricing |
| `web/dashboard.html` | Owner dashboard (login, analytics, KB manager, ticket table, chat tester) |
| `web/widget.html` | Embeddable-widget demonstration on a fictional storefront |
| `README.md` / `BUSINESS.md` | Product overview / business & service pack |
| `results.json` / `triage_model.npz` | Measured metrics / trained model artefact |

## 3.5 Design decisions and their reasons

1. **Stdlib-only server.** Zero dependencies means zero install failures on a client's or buyer's machine, instant auditability, and ₹0 serving cost beyond the VPS. The cost — hand-written routing and JSON handling — is small at this scale and is paid once.
2. **Extractive answers, not generated ones.** Returning the article verbatim makes hallucination structurally impossible in the answer path and makes the citation trivially truthful. Generation is a roadmap upgrade *behind the same gate* (Chapter 9).
3. **The gate is a single named constant.** `CHAT_THRESHOLD = 0.12` is one line, tunable per deployment philosophy: raise it and the bot answers less but errs less; lower it and deflection rises at the risk of weak matches. Making the product's central risk dial a named, documented constant is deliberate.
4. **Triage runs on *every* message, answered or not.** Intent breakdown then describes all demand, not just failures — the owner learns what customers ask about even when the bot copes.
5. **Tests run the real code paths.** `tests.py` imports `app` against a temporary database and calls the same functions the HTTP layer calls; `smoke_test.py` goes further and speaks real HTTP to a real server process. There is no mock layer to hide behind.

# Chapter 4 — Implementation: Every Feature and Every Function

This chapter walks the codebase file by file. Function signatures are given as they appear in the source; together they are the complete functional surface of the product.

## 4.1 `ml.py` — the triage engine

### 4.1.1 Label spaces and data templates

```
INTENTS    = ["billing", "technical", "delivery", "refund", "account", "general"]
PRIORITIES = ["low", "normal", "high", "urgent"]
```

`_TEMPLATES` holds 8 hand-written message templates per intent (48 in total), each written in realistic customer voice and parameterised with `{amt}` (₹499 / ₹1,299 / ₹2,999 / ₹750) and `{oid}` (#UK10231, #UK88410, #OD5521, #IN90210) placeholders. `_PRIORITY_PHRASES` holds 4 suffix phrases per priority — from low's "No hurry, whenever you get time." to urgent's "URGENT: this is an emergency, I need help immediately!" — with "normal" including the empty string (most real messages carry no explicit priority marker, and the model must learn priority from content alone in that case). `_fill(tpl, rng)` substitutes the placeholders from a seeded RNG.

### 4.1.2 `generate_data(n_per_combo=25, seed=42)`

Builds the corpus as the Cartesian product intent × priority: for each of 6 intents × 4 priorities, 25 messages are drawn (random template + random suffix + random fillers) **600 labelled examples**, shuffled. Because priority is carried by the *suffix* and intent by the *template body*, the two heads learn genuinely different signals — intent from topical vocabulary, priority from urgency vocabulary — which is precisely the structure the evaluation in Chapter 7 probes.

### 4.1.3 `tokenize(text)`

`re.findall(r"[a-z0-9]+", text.lower())` — lowercases and extracts alphanumeric runs. One tokeniser is shared by the classifier, the sentiment lexicon, the urgency rule and (via `_rtok`) retrieval, so every component sees identical tokens; tokenisation drift between training and serving is impossible by construction.

### 4.1.4 `class Tfidf(max_vocab=3000)`

`fit(docs)` — computes document frequencies, keeps the 3,000 most frequent terms as the vocabulary, and stores smoothed IDF weights `ln((1+n)/(1+df)) + 1`.

`transform(docs)` — builds the count matrix, applies `(1 + ln count) · idf`, and L2-normalises each row (zero rows guarded). Output: dense NumPy array, documents × vocabulary.

### 4.1.5 `class SoftmaxLR`

`__init__(n_features, n_classes, seed=0)` — `W ~ N(0, 0.01)`, `b = 0`.

`fit(X, y, epochs=300, lr=0.5, l2=1e-4)` — full-batch gradient descent on cross-entropy with the max-subtraction stability trick and L2 shrinkage, exactly as derived in §2.4.

`predict_proba(X)` — stable softmax probabilities.

#### 4.1.6 `sentiment(text)`, `macro_f1(y_true, y_pred, labels)`

`sentiment()` counts lexicon hits and returns `positive` / `negative` / `neutral` by majority (§2.5). `macro_f1()` computes per-class precision/recall/F1 from raw label arrays with zero-division guards and averages them — the evaluation metric of Chapter 7, implemented in 10 lines with no sklearn.

#### 4.1.7 `class TriageModel`

`train(data)` — fits the shared TF-IDF on all texts, then trains two independent `SoftmaxLR` heads (seeds 1 and 2) on the intent and priority label indices.

`evaluate(data)` — returns `{intent_accuracy, intent_macro_f1, priority_accuracy, priority_macro_f1, n_test}` from argmax predictions. This is the sole source of the numbers in `results.json`.

`predict(text)` — the serving path. Vectorises the message, takes both heads' argmax and max-probability confidences, then applies the **urgency rule layer**: if any token is in `URGENCY_SIGNALS = {urgent, urgently, emergency, asap, immediately, critical}`, priority is forced to `urgent` and `priority_source` becomes `"rule_override"` (otherwise `"ml"`). Returns `{intent, intent_confidence, priority, priority_confidence, priority_source, sentiment}`.

`save(path)` / `load(path)` — serialises vocabulary (as JSON), IDF vector and both heads' `W`/`b` into one `.npz`; `load` returns `None` if the file is absent, which `app.py` treats as "train on the fly" (see `get_model()`).

## 4.2 `train_ml.py` — the training run

`main()` performs the experiment of Chapter 6 in 30 lines: generate 600 examples; group them by (intent, priority) pair; shuffle within each group with RNG seed 7 and split 80/20 **480 train / 120 test**, stratified by construction so every class pair appears in both sets in proportion; time `TriageModel().train(train)` with `time.time()`; evaluate on the held-out test; save the model and write `results.json` with dataset size, train size, wall-clock training seconds, the four metrics, test size and model filename. Anyone can reproduce the exact numbers in this report with `python3 train_ml.py`.

## 4.3 `app.py` — the product server

### 4.3.1 Model lifecycle — `get_model()`

A module-level lazy singleton: the first chat request loads `triage_model.npz`; if the file is missing, it generates data and trains in-process, so the server is self-healing on a fresh checkout. `CHAT_THRESHOLD = 0.12` lives beside it as the product's central constant (§3.5).

### 4.3.2 Persistence — `db()`, `init_db()`

`db()` opens a SQLite connection with `row_factory = sqlite3.Row` (dict-style rows) — a fresh short-lived connection per operation, which keeps the threaded server free of shared-connection locking concerns at MVP scale. `init_db()` creates the five tables of §3.3 with `CREATE TABLE IF NOT EXISTS`, safe on every boot.

### 4.3.3 Authentication — `_hash()`, `register_tenant()`, `login()`, `tenant_for_token()`, `tenant_for_widget_key()`

`_hash(pw, salt)` — PBKDF2-HMAC-SHA256, 60,000 iterations, hex digest (§2.6).

`register_tenant(name, industry, email, password)` — rejects duplicate emails ("email already registered"), generates salt and widget key (`wk_` + `token_hex(10)`), inserts the tenant, returns `{id,`

`name, widget_key}`.

`login(email, password)` — looks up by email, recomputes the hash with the stored salt, compares; on success mints `token_hex(16)`, stores the session, returns the token; on any failure returns `None` (the HTTP layer turns it into a uniform 401 "invalid credentials", leaking no hint about which field failed).

`tenant_for_token(token)` — one JOIN from `sessions` to `tenants`; `None` for missing/unknown tokens. Every owner endpoint funnels through the handler's `_auth()` wrapper around this function.

`tenant_for_widget_key(key)` — the chat path's tenant resolution.

### 4.3.4 Knowledge base — `add_kb()`, `list_kb()`

Thin, tenant-scoped inserts and ordered selects. There is intentionally no update-in-place: owners delete and re-add, keeping the MVP's mutation surface minimal and auditable (deletion is implemented in the HTTP layer's `do_DELETE`, scoped by `id AND tenant_id`).

### 4.3.5 Retrieval — `_rtok()`, `retrieve(tenant_id, query)`

`_rtok()` is the retrieval tokeniser: shared `tokenize()`, minus the 40-word `_STOP` list, minus a trailing plural "s" for words longer than 3 characters. `retrieve()` implements §2.2 exactly: corpus = `title + " " + title + " " + body` per article (title double-weighted); document frequencies over this tenant's articles only; log-TF × smoothed-IDF vectors for query and each article; cosine against every article; returns `(best_doc, float_score)`, or `(None, 0.0)` for an empty KB. Because `df` and the candidate set are built from `list_kb(tenant_id)`, cross-tenant retrieval is impossible by construction — and is also test-verified (§6.2).

### 4.3.6 The chat gate — `chat(widget_key, message)`

The product's defining function (§3.2). Order of operations matters: tenant resolution first (invalid key  error, nothing logged); triage second (so even ticketed conversations are classified); retrieval third; then the branch. The answering branch returns `{answer, citation, confidence, ticket_created: False, ticket_id: None, **triage}`. The abstaining branch inserts the conversation with `resolved_by_ai = 0`, inserts the ticket with status `open` referencing the conversation, commits both in one transaction sequence, and returns the same shape with `ticket_created: True` and the real `ticket_id`. The fixed abstention message — "Thanks for reaching out. I couldn't find a confident answer in our help articles, so I've created a support ticket — our team will get back to you." — is written to inform the customer of the handoff, never to disguise it.

### 4.3.7 Tickets — `list_tickets()`, `set_ticket_status()`

`list_tickets()` fetches the tenant's tickets and sorts in Python by `(priority_rank, -created)` with rank urgent=0, high=1, normal=2, low=3 — urgency first, newest first within a priority. `set_ticket_status()` whitelists the three legal statuses and scopes the `UPDATE` by tenant, returning whether a row actually changed — which is what makes the owner-only test (§6.2, test 13) meaningful.

### 4.3.8 Analytics — `analytics(tenant_id)`

Three SQL aggregates over the tenant's rows: conversation count, AI-resolved sum and mean confidence in one query; ticket count; intent counts via `GROUP BY`. Returns `{total_conversations, ai_resolved, deflection_rate (rounded, 0.0 when total is 0), tickets_created, avg_confidence, intent_breakdown}`. No caching, no estimates — the dashboard always shows the database's current truth.

### 4.3.9 Demo seeding — `seed()`, `URBANKART_KB`, `FITLIFE_KB`

`seed()` runs at boot only if the tenants table is empty. It registers two fictional businesses through the *same* `register_tenant()` path a real customer uses, then loads their articles through `add_kb()`: **UrbanKart** (e-commerce; 10 articles — order tracking, delivery charges/timelines, returns & refunds, cancellation, payment methods, electronics warranty, address change, damaged/wrong product, offers & loyalty, account/login help) and **FitLife Gym** (fitness; 5 articles — plans & pricing ₹1,500/₹3,900/₹12,000, membership freeze, class schedule, refund/cancellation, timings). Both use password `demo1234` (emails `demo@urbankart.in`, `demo@fitlife.in`) and the printed widget keys make the demo fully explorable. The seed corpora are realistic enough that the widget demo answers real questions — and, importantly, refuses ones outside the articles.

### 4.3.10 HTTP layer — `class Handler`

A `BaseHTTPRequestHandler` subclass (`log_message` silenced) with helpers `_send(code, body, ctype)` (JSON by default, raw bytes/HTML for pages), `_body()` (JSON request parsing), `_auth()` (Bearer extraction `tenant_for_token`), and `_static(name)` (serves files from `web/`, 404 if absent). Routing:

> **GET** `/` landing, `/dashboard` dashboard, `/widget-demo` widget; `/api/me`, `/api/kb`, `/api/tickets`, `/api/analytics` (all 401 without a valid token); `/api/demo-tenants` (public, demo builds only — lists tenant names/industries/widget keys so the demo widget can find UrbanKart).
>
> **POST** `/api/register` (201, or 400 with the error), `/api/login` (200 + token, or 401), `/api/chat` (200, or 404 for an invalid widget key), `/api/kb` (201, owner only), `/api/tickets/<id>/status` (regex-matched; 200 `{ok: true}` or 404) — malformed JSON bodies are a clean 400.
>
> **DELETE** `/api/kb/<id>` (owner only, tenant-scoped).

`main()` runs `init_db()`, `seed()`, then serves `ThreadingHTTPServer` on `0.0.0.0:$PORT` (default 8090).

## 4.4 `web/index.html` — the landing page

Single-file page (inline CSS, indigo design system) with: sticky header (logo + Owner Dashboard and Live Widget Demo buttons); hero — "Your support inbox, answered by AI in seconds." with the value proposition and a CTA into the live demo; four feature cards (Instant KB answers, Smart ticket triage, Deflection analytics, Multi-business ready) mirroring the implemented features one-to-one; a three-tier pricing grid (Starter ₹4,999/mo, Growth ₹9,999/mo marked "Most popular", Pro ₹19,999/mo) with the ₹14,999 one-time setup line beneath; and a footer that honestly labels the demo build ("lexical retrieval + ML triage, no external APIs"). The marketing copy makes no claim the code does not keep — a deliberate product-ethics choice that Chapter 8's sales pitch maintains.

## 4.5 `web/dashboard.html` — the owner dashboard

One page, vanilla JS against the API, token persisted in `localStorage` (`rd_token`):

> **Login box** (prefilled with the UrbanKart demo credentials) — `doLogin()` posts to `/api/login`, stores the token, and calls `refresh()`.

**Analytics card** — five stat blocks (total conversations, AI-resolved, deflection % , tickets, average confidence) plus the raw intent-breakdown object, all filled by `refresh()` from `/api/analytics`.

**Knowledge-base card** — title/body inputs; `addKb()` posts to `/api/kb`; the article list re-renders from `/api/kb`.

**Tickets table** — ID, message (truncated to 60 chars), intent, priority (pill-styled), sentiment, status — rendered in the server's urgent-first order from `/api/tickets`.

**Live chat tester** — shows the tenant's widget key; `sendChat()` posts to `/api/chat` and prints the **full raw JSON response** (answer, citation, confidence, triage, ticket flag) — the exact tool the 48-hour onboarding playbook (Chapter 8) uses to tune a client's KB against their 50 most common questions.

`refresh()` also calls `/api/me` to display the business name and widget key — the snippet the owner pastes into their site.

## 4.6 `web/widget.html` — the embeddable widget, demonstrated

A fictional "UrbanKart" storefront page (orange theme, deliberately distinct from ResolveDesk's indigo, to show the widget lives on the *client's* brand) with a fixed bottom-right chat card: greeting, scrolling log, input row. On load it fetches `/api/demo-tenants`, finds UrbanKart's widget key, and `send()` posts each message to `/api/chat`, rendering the answer with its `[Source: …]` citation line, and — when a ticket is created — the ticket number with its intent and priority, so a prospect watching the demo sees the refusal path as a feature, not a failure. In a client deployment this same card is the embed: the key is the only per-client configuration.

---

# Chapter 5 — Data: The Triage Corpus and the Seed Knowledge Bases

## 5.1 The synthetic triage corpus

The classifier's dataset is generated, not collected, by `ml.py: generate_data()` (protocol in §4.1.2):

| Property | Value |
|---|---|
| Total examples | 600 |
| Structure | 6 intents × 4 priorities × 25 |
| Intent templates | 8 per intent (48 total), parameterised by amount and order-ID fillers |
| Priority signal | 4 suffix phrases per priority (16 total; "normal" includes the empty suffix) |
| Split | Stratified by (intent, priority): 480 train / 120 held-out test |
| Seed | 42 (generation), 7 (split shuffling) |

**Why synthetic, stated plainly.** An MVP built before its first client has no real ticket history to train on; the alternatives were to ship no ML at all or to bootstrap on generated data and re-tune per client. The project chose the second, and labels the consequence wherever the metrics appear: template-generated text is far more regular than real customer language (fixed phrasings, clean spelling, no code-switching, no sarcasm), so held-out performance on it is an **upper bound**, useful for verifying that the pipeline, the heads and the metrics are correct — not for promising a client 100% of anything. Chapter 8's onboarding playbook therefore includes the corrective step: triage is re-tuned on the client's real ticket exports during setup, and the same `train_ml.py` + `evaluate()` path re-measures it on their data.

**What the corpus does test honestly.** Because intent lives in the template body and priority lives in the appended phrase, the corpus separates the two learning problems. A model cannot ace priority by memorising intent vocabulary; it must find the urgency signal — including learning, from the empty-suffix normal examples, when priority must be inferred from content. The residual priority errors in Chapter 7 occur exactly at this designed difficulty.

## 5.2 The seed knowledge bases

The demo tenants' articles (§4.3.9) double as the product's retrieval test fixtures: UrbanKart's 10 articles cover the canonical e-commerce question set, FitLife's 5 cover a local-service business. Their bodies are written as complete, self-contained answers (prices, day counts, conditions) because the chat engine returns them verbatim — KB quality *is* answer quality in an extractive design, a fact the onboarding playbook operationalises by writing/loading articles with the client. The test suite adds a third, purpose-built article ("Refund policy") to verify retrieval precision against a known target.

# Chapter 6 — Experiments and Product QA Protocol

A product's evidence comes in two kinds: **model metrics** (does the classifier work?) and **product QA** (does the system work, end to end, including the parts that must *not* happen?). This chapter specifies both protocols completely; Chapter 7 reports the outcomes.

## 6.1 Triage classifier experiment

| Setting | Value |
|---|---|
| Features | TF-IDF, max vocabulary 3,000, log-TF, smoothed IDF, L2-normalised |
| Model | Softmax logistic regression ×2 heads (intent: 6 classes; priority: 4) |
| Optimiser | Full-batch gradient descent, 300 epochs, lr 0.5, L2 λ = 1e-4 |
| Initialisation | W ~ N(0, 0.01) (seeds 1, 2), b = 0 |
| Data | 600 generated; stratified 480 train / 120 test (seed 7) |
| Metrics | Accuracy and macro-F1 per head (`TriageModel.evaluate()`) |
| Timing | Wall-clock training time on CPU, recorded in `results.json` |
| Command | `python3 train_ml.py` |

There is no hyperparameter sweep to report, and this report does not invent one: the configuration above is the single configuration run, chosen a priori as standard for a small linear text model, and it is sufficient for the pipeline-verification purpose of §5.1. The honest "experiment" here is the held-out split itself — training and test messages share templates but differ in sampled fillers, phrases and combinations, so the metrics measure generalisation across the generator's variation, not memorisation of specific strings (though, per §5.1, they cannot measure generalisation to real human phrasing).

## 6.2 The functional test suite — `tests.py` (13 tests)

The suite points `RESOLVEDESK_DB` at a fresh temporary directory, imports `app`, initialises the schema, and asserts, in order:

| # | Test | What it proves |
|---|---|---|
| 1 | register two tenants | Registration works; widget keys are distinct |
| 2 | login works / bad login rejected | Correct password mints a token; wrong password returns `None` |
| 3 | KB add | Article inserts and lists (count = 1) |
| 4 | KB retrieve finds refund article | Retrieval ranks the right article at/above the 0.12 threshold |
| 5 | chat answers known question | The gate's answering branch fires; no error |
| 6 | chat cites the article | Citation equals the article title exactly |
| 7 | low-confidence question creates a ticket | The abstention branch fires ("chartered helicopter delivery to the moon"); ticket count increments by exactly 1 |
| 8 | triage: refund intent | `predict()` classifies a refund message as `refund` |
| 9 | triage: urgent priority | The same message ("...this is urgent!") yields priority `urgent` |

| # | Test | What it proves |
|---|---|---|
| 10 | analytics counts match actions | After the scripted actions: 2 conversations, tickets = prior + 1, AI-resolved = 1 |
| 11 | tenant isolation: B cannot retrieve A's KB | Tenant B has no articles and retrieval over B returns None for A's question |
| 12 | tenant isolation: B has no tickets | B's ticket list is empty; B's analytics show 0 conversations |
| 13 | ticket status update (owner only) | A can resolve its own ticket; B attempting to mutate A's ticket changes nothing |

Tests 11–13 are the multi-tenancy guarantee of Objective O1 expressed as executable assertions; test 7 is the anti-hallucination guarantee of O2; test 10 ties the dashboard's numbers to ground truth (O4).

## 6.3 The HTTP smoke test — `smoke_test.py` (10 steps)

Where `tests.py` exercises functions, the smoke test exercises the **deployed artefact**: it starts `app.py` as a subprocess on port 8199 against a temp DB, waits for readiness, and performs the real user journey with `urllib` calls:

| # | Step | Assertion |
|---|---|---|
| 1 | landing page served | GET `/` 200, contains "ResolveDesk AI" |
| 2 | dashboard served | GET `/dashboard` 200, contains "Dashboard" |
| 3 | widget demo served | GET `/widget-demo` 200, contains "UrbanKart" |
| 4 | register tenant | POST `/api/register` 201, widget key starts `wk_` |
| 5 | login | POST `/api/login` 200 with a token |
| 6 | add KB article | POST `/api/kb` (Bearer) 201 |
| 7 | chat answers from KB with citation | "What are your store timings?" citation "Store timings", no ticket |
| 8 | unknown question creates ticket | "Can I book a helicopter tour?" `ticket_created` true |
| 9 | analytics reflect journey | 2 conversations, 1 ticket, 1 AI-resolved |
| 10 | ticket listed | GET `/api/tickets` returns exactly the created ticket |

This is the exact script a prospective client or employer can run to verify the product works on their own machine in under a minute.

---

# Chapter 7 — Results and Analysis

## 7.1 Triage classifier — measured results (`results.json`)

| Metric | Result |
|---|---|
| Dataset size | 600 messages |
| Train / held-out test | 480 / 120 |
| **Intent accuracy** | **1.000 (100%)** |
| **Intent macro-F1** | **1.000** |
| **Priority accuracy** | **0.9833 (98.3%)** |
| **Priority macro-F1** | **0.9833** |
| Training time | 14.12 s on CPU |
| Model artefact | `triage_model.npz` |

## 7.2 Product QA — measured results

| Suite | Result |
|---|---|
| `tests.py` (functional + isolation) | **13 / 13 passed** ("ALL 13 TESTS PASSED") |
| `smoke_test.py` (real HTTP journey) | **10 / 10 steps passed** ("SMOKE TEST PASS — 10 steps") |

Both suites were re-run for this report against the delivered code; the outputs above are their actual printed results.

## 7.3 Analysis

**Intent is, on this corpus, a solved sub-problem — and the reason is instructive.** The six intents occupy almost disjoint vocabularies (invoice/charge; crashes/error; order/delivered; refund/returned; password/account; hours/store). A linear model over TF-IDF features is exactly matched to such structure, so 100% held-out accuracy with macro-F1 1.000 is credible *for this generator* — and simultaneously the clearest sign of the corpus's regularity (§5.1). Real tickets mix intents ("my order is late and I want a refund"), misspell, and switch languages; the production answer to that is per-client re-tuning and, at the limit, the embedding/LLM upgrade path — not a bigger claim for this number.

**Priority is the genuinely harder head, and its 98.3% is the more interesting result.** 98.3% on 120 test messages corresponds to 2 misclassified messages; macro-F1 0.9833 tracking accuracy closely shows the errors are not concentrated in one ignored class. The designed difficulty (§5.1) explains where the residual error lives: messages whose priority phrase is empty or mild must be graded from content alone, and the boundary between "high" and "urgent" phrasing is a matter of degree, not kind. This is also where the architecture hedges correctly: the cases a business most fears — explicit emergency vocabulary — are removed from the ML head's jurisdiction entirely by the urgency rule layer, so the *operational* cost of the residual 1.7% falls on the low-stakes boundary (low/normal/high ordering), not on missed emergencies. Test 9 (§6.2) verifies the combined behaviour end to end.

**Training cost is a product feature.** 14.12 seconds on CPU for the full model means per-client re-tuning during onboarding is effectively free and instantaneous at delivery scale — no GPU, no

training bill, no waiting. The 25 KB model file loads at server start with no perceptible delay. In the unit economics of Chapter 8, this is why the current stack's marginal cost per client is only hosting and the author's tuning time.

**The QA results carry the product claims the classifier metrics cannot.** A classifier score says nothing about whether Tenant B can read Tenant A's tickets; tests 11–13 do. A README claim of "cited answers" is marketing; tests 5–6 and smoke step 7 are evidence. The report's position is that for a *product*, this second table is at least as important as the first — it is what converts a demo into something sellable, and it is the evidence base for the money-back setup promise in the sales pitch (§8.5).

**Failure cases and honest edges.** (i) Retrieval: a question phrased with vocabulary disjoint from every article scores below 0.12 and becomes a ticket even when a human could map it to an article — the safe direction, at a deflection cost; the onboarding tuning loop exists precisely to close such gaps with better articles. (ii) Sentiment: the lexicon misreads negation ("not happy" counts positive "happy") and domain words ("refund" is auto-negative even in a neutral refund-status question) — acceptable for a dashboard signal, disclosed here. (iii) Priority: as analysed above. (iv) The demo-tenants endpoint is public by design for the demo and must be removed or authenticated in a paid deployment (Chapter 9).

# Chapter 8 — The Business: Market, Pricing, Unit Economics, Delivery

*Source: `BUSINESS.md`, the project's own sell-it-and-deliver-it playbook. Financial figures in §8.3 are the playbook's estimates and are labelled as such; they are not measured results.*

## 8.1 What is being sold

Not software — an **outcome, installed**: an AI support assistant on the client's own website that answers from the client's FAQs/policies, creates and prioritises tickets when it cannot answer, and gives the owner a dashboard proving how much support was deflected. Sold as a done-for-you service plus subscription, because the target customer has no one to "administer a platform" — the configuration *is* the service.

## 8.2 Target market

D2C / e-commerce stores (Shopify/WhatsApp sellers) drowning in "where is my order / refund?" messages;

Local service businesses: gyms, clinics, coaching institutes, salons, repair shops;

Small SaaS / ed-tech teams with repetitive support queries.

Sweet spot: **200–5,000 support conversations per month, no dedicated support team** — large enough to feel the pain daily, small enough that enterprise per-seat pricing makes no sense.

## 8.3 Pricing and unit economics

| Tier | Price | Includes |
|---|---|---|
| Starter | ₹4,999/mo (~$59) | 1 business, 500 conversations/mo, KB chat widget, ticket triage |
| Growth | ₹9,999/mo (~$119) | 2,000 conversations/mo, analytics, monthly KB tuning, priority support |
| Pro | ₹19,999/mo (~$239) | Unlimited conversations, multiple brands, custom integrations |
| One-time setup | ₹14,999 (~$179) | KB loading, widget branding + install, triage tuning on client's real tickets, staff training call |

**Unit economics (ESTIMATE — labelled, not measured):** hosting per client on a small shared VPS ₹800–1,500/mo; maintenance/tuning  2 hrs/mo of the author's time. At the Growth tier,  ₹9,999 revenue  ₹1,200 infrastructure  **~₹8,800/mo gross per client before time**; eight Growth clients  ₹70k/mo recurring — in line with the author's ₹60,000+/month income target. The estimates hold because the current stack has **zero API cost** (stdlib + NumPy; the 14-second CPU retrain of §7.3 is the whole compute story). The playbook notes the pricing consequence honestly: adding a paid LLM backend adds roughly ₹1–3 per conversation in API cost and must be priced into the Pro tier.

## 8.4 The 48-hour onboarding playbook (freelance delivery)

**Day 0 (30-minute call):** collect FAQs, policies, price lists, past support chats (WhatsApp/email exports); agree escalation rules.

**Hours 0–6:** create the tenant, load KB articles, paste the widget snippet into the client's site (or hand it to their developer), brand colours/logo.

**Hours 6–24:** tune — run the client's 50 most common real questions through the dashboard's live chat tester (§4.5); add/adjust articles until deflection on that set is satisfactory; verify urgent messages escalate (the rule layer of §4.1.7 makes this check quick and deterministic).

**Hours 24–48:** owner training call (dashboard, tickets, adding articles), go live, hand over a one-page SOP; schedule the first-week check-in.

Every step maps to an implemented feature — the tester, the KB manager, the ticket view — which is why 48 hours is a credible promise rather than a slogan.

## 8.5 The one-page sales pitch (as written in BUSINESS.md)

> ***Your customers ask the same 20 questions every day. ResolveDesk AI answers them in seconds — and only bothers you with the rest, already sorted by urgency.*** *Installed on your website in 48 hours, trained on your policies (not generic AI guesses — when it doesn't know, it creates a ticket instead of making things up). You get a dashboard proving how many conversations it resolved without you. ₹4,999/month, set up for you. First demo runs on your own FAQs — if it can't answer them, you don't pay the setup fee.*

Note how the pitch sells the engineering: the confidence gate *is* the trust argument, the dashboard *is* the proof argument, and the demo-on-your-own-FAQs offer is underwritten by the smoke-test discipline of §6.3.

## 8.6 Competitor positioning (honest)

| | Intercom Fin / Zendesk AI | ResolveDesk AI |
|---|---|---|
| Pricing | Per-seat + per-resolution, USD, adds up fast for SMBs | Flat ₹/mo, SMB-first |
| Setup | Self-serve; client configures everything | Done-for-you in 48 hours |
| Best for | Larger teams with support staff | Owner-run businesses with no support team |
| Where they win | Deeper integrations, mature LLM answers, enterprise security | — no parity is claimed; the roadmap (Chapter 9) closes the answer-quality gap |

## 8.7 Why the product and the portfolio reinforce each other

For freelance work, ResolveDesk is a *productised service*: a fixed-scope, fixed-price offer with a working demo, a delivery playbook and recurring revenue, instead of hourly labour. For regular-job applications, the same artefact evidences end-to-end ownership — ML, backend, frontend, testing, documentation and commercial thinking — in one repository. Chapter 11's Q&A prepares the author to defend it in both rooms.

---

# Chapter 9 — Discussion: Limitations, Threats to Validity, Production Path

## 9.1 Limitations (as documented in the project itself)

1. **Demo-grade authentication.** PBKDF2 hashing and session tokens are real, but there is no token expiry, no rate limiting, and the server speaks HTTP, not HTTPS. Suitable for a demo on a local network; not for handling real customer PII in production.
2. **Lexical retrieval ceiling.** TF-IDF cannot bridge vocabulary gaps (§7.3). The gate converts this weakness into tickets rather than errors — safe, but it caps deflection until the embedding/LLM upgrade lands.
3. **Synthetic triage data.** The headline metrics are an upper bound (§5.1). The mitigation is procedural (per-client re-tuning) and is built into the paid setup.
4. **SQLite scale ceiling.** One file, short-lived connections: correct and simple at MVP volume; Postgres is the growth path.
5. **No payments or quota enforcement yet.** Pricing exists as a business artefact; Razorpay/Stripe integration and per-plan conversation quotas are roadmap items, so the current build cannot self-serve paying tenants.
6. **Demo-only public endpoint.** `/api/demo-tenants` exposes widget keys for the demo; widget keys are routing credentials (§2.1), but the endpoint should still be removed outside demo deployments.

## 9.2 Threats to validity

The classifier evaluation reuses the generator's templates across train and test (with different sampled fillers/phrases), so it validates the *pipeline* more than real-world accuracy — the report has scoped every claim accordingly. The QA suites validate behaviour on scripted journeys; they do not fuzz inputs, load-test the threaded server, or test browser compatibility beyond the pages' own simple JS. Analytics correctness is verified for the scripted counts, not for concurrent-write races at scale. None of these gaps is hidden; each has a named remediation in §9.3.

## 9.3 Production roadmap (from BUSINESS.md)

1. **LLM answer layer** (Groq/OpenRouter) behind the *same* citation + confidence gate, lexical engine as fallback — answer quality rises, the safety property is unchanged.
2. **WhatsApp channel** via Meta Cloud API — the channel Indian SMB customers actually use.
3. **Real auth & deployment hardening** — OTP/OAuth, rate limiting, HTTPS, backups; remove the demo endpoint.
4. **Payments + plan enforcement** — Razorpay/Stripe, conversation quotas per tier.
5. **Per-client triage training** on real ticket history, with a human-in-the-loop review queue for low-confidence classifications.
6. **Hindi/Hinglish KB support** — tokenisation and templates extended for the market's actual language mix.

The order is deliberate: answer quality and channel first (client-visible value), hardening alongside the first paying deployments, monetisation plumbing when there is something to charge for.

# Chapter 10 — Conclusion and Future Work

ResolveDesk AI set out to be the thing the other projects in this portfolio are not: not a research result, but a **product a small business can use on the day it is installed, and a service its author can sell**. Against its six objectives: multi-tenancy is implemented and isolation-tested (O1); answers are grounded, cited, and gated, with an honest ticket path below threshold (O2); every conversation receives ML triage with a hybrid urgency override (O3); the dashboard reports only real database aggregates, including the deflection rate that is the product's commercial proof (O4); the stack is stdlib + NumPy with one-command train/serve and two passing automated suites — 13 functional tests and a 10-step HTTP journey (O5); and the repository carries its own business playbook — pricing, estimated unit economics, a 48-hour delivery process, an honest competitor table and a roadmap (O6).

The measured results are strong where the design says they should be and modest where honesty requires: intent classification at 100% / priority at 98.3% on a synthetic held-out set that this report repeatedly scopes as an upper bound; QA at 13/13 and 10/10 on the real code paths. Future work is the roadmap of §9.3 — above all the gated LLM answer layer and the WhatsApp channel — plus the first real-client deployment, whose ticket history will replace the synthetic corpus and turn this report's caveated metrics into field numbers.

# Chapter 11 — Learning Guide

## 11.1 Concept checklist

A reader who can explain each item below understands this product:

- [ ] Multi-tenancy with a discriminator column, and why tenant scope belongs in every query's `WHERE` clause
- [ ] The difference between a routing credential (widget key) and an authorisation credential (session token)
- [ ] TF-IDF: log-scaled TF, smoothed IDF, and why cosine similarity is length-invariant
- [ ] Stopword removal, light stemming, and title field-weighting as retrieval refinements
- [ ] Confidence gating / selective prediction, and why error costs are asymmetric in support
- [ ] Deflection rate as a product metric, computed from an append-only conversation log
- [ ] Softmax (multinomial logistic) regression: logits, cross-entropy, the `(P − Y)` gradient, L2, numerical stability
- [ ] Accuracy vs macro-F1, and why rare-but-critical classes demand macro averaging
- [ ] Lexicon sentiment: strengths (transparent, free) and failure modes (negation, domain words)
- [ ] Hybrid rule+ML systems: when a deterministic override beats a probability, and auditing overrides
- [ ] Salted PBKDF2 password storage and session-token auth; what demo-grade auth still lacks
- [ ] The difference between function-level tests and an end-to-end HTTP smoke test, and why a product needs both
- [ ] Productised services: setup fee + subscription, onboarding as part of the product, unit economics

## 11.2 Interview / client Q&A

**Q1. Why does the bot sometimes refuse to answer? Isn't that a failure?** It is the central design decision. Answering requires a KB article at cosine  0.12; below that, the bot creates a triaged ticket and tells the customer so. A fabricated refund promise costs the business money and trust; a ticket costs a human a few minutes on the normal path. The refusal is what makes the answers trustworthy — and the dashboard shows the resulting deflection rate so the trade-off is measured, not guessed.

**Q2. How do you guarantee one business cannot see another's data?** Three layers: tenant identity is derived server-side from the session token or widget key, never from the request body; every read and mutation is scoped by `tenant_id` in SQL, so even a guessed article or ticket ID from another tenant matches zero rows; and tests 11–13 of `tests.py` assert isolation directly — B cannot retrieve A's articles, sees none of A's tickets or analytics, and cannot change A's ticket status.

**Q3. Why TF-IDF instead of embeddings or an LLM?** Cost, determinism and deployability for the MVP: zero API cost per conversation, no keys, fully offline, and every score explainable. The architecture isolates retrieval behind one function, so embeddings or an LLM layer can replace it behind the same citation + confidence gate — roadmap item 1 — without touching tickets, triage or

analytics.

**Q4. Explain the triage model.** TF-IDF features (vocabulary 3,000) feed two softmax logistic-regression heads trained from scratch in NumPy: intent over 6 classes, priority over 4. Softmax turns logits into probabilities; training is full-batch gradient descent on cross-entropy (gradient `(P − Y)/n`) with L2 1e-4, 300 epochs. A lexicon adds sentiment, and a rule layer forces priority to urgent on explicit emergency vocabulary, recording `priority_source` so overrides are auditable.

**Q5. Your intent accuracy is 100% — should I believe it?** On the held-out split of the synthetic corpus, yes — that is what `results.json` records, and `train_ml.py` reproduces it. As a claim about real customer messages, no: the corpus is template-generated, so intents are vocabulary-disjoint in a way real messages are not. The report labels the number an upper bound, and the delivery process re-tunes and re-measures the model on each client's real ticket history.

**Q6. Why is priority accuracy (98.3%) lower, and does it matter?** Priority is a matter of degree — the high/urgent boundary is genuinely fuzzy, and "normal" messages often carry no explicit marker, so 2 of 120 test messages land on the wrong side of a neighbouring class. Operationally it matters least where it counts: explicit emergencies are escalated by rule, not left to the model, so residual ML error affects ordering among non-emergencies.

**Q7. What is the deflection rate and why do clients care?** AI-resolved conversations divided by total conversations, computed from the conversation log. It is the product's ROI in one number: each deflected conversation is a question the owner never had to answer. It is also the tuning target during onboarding — the 50-question tuning loop raises it by improving the KB, not by loosening the gate.

**Q8. How is authentication implemented, and what would you harden first?** Passwords are stored as PBKDF2-HMAC-SHA256 with per-user salts (60,000 iterations); login mints a 128-bit session token. It's honestly labelled demo-grade: first production steps are HTTPS, token expiry, and rate limiting, then OTP/OAuth — roadmap item 3.

**Q9. How would this scale to 100 clients?** The stateless threaded server scales horizontally behind any reverse proxy; the real change is SQLite  Postgres and moving sessions with it. Per-client model re-tuning is 14 seconds of CPU, so even per-tenant models are feasible. WhatsApp and payments (roadmap items 2, 4) matter commercially before raw scale does.

**Q10. What does the client actually pay, and what do you earn?** ₹14,999 one-time setup plus ₹4,999/₹9,999/₹19,999 per month by tier. Estimated unit economics (labelled estimates in BUSINESS.md): roughly ₹1,200/mo infrastructure per Growth client leaves ~₹8,800/mo gross before the author's ~2 hours of monthly tuning time; eight Growth clients  ₹70k/mo recurring. Zero API cost in the current stack is what makes the margin work.

**Q11. How is this different from Intercom Fin or Zendesk AI?** Audience and model, not a claim of parity: they are per-seat/per-resolution, self-serve platforms for teams with support staff, and they win on integrations, LLM answer maturity and enterprise security. ResolveDesk is flat-₹, done-for-you in 48 hours, for owner-run businesses with no support team. The report's competitor table says exactly that.

**Q12. If you had one more week, what would you build?** The gated LLM answer layer (roadmap 1): embedding retrieval + generated answers that must cite a retrieved article and pass the same confidence gate, with the current lexical engine as fallback — because it raises deflection without surrendering the no-hallucination property that the whole product is sold on.

## 11.3 Exercises to extend the project

1. Add a third tenant with a Hindi-language KB; observe where `tokenize()` and the stopword list break, and fix them.
2. Lower/raise `CHAT_THRESHOLD` to 0.05 and 0.25; using `tests.py` scenarios, tabulate answers-vs-tickets and argue for a per-tenant threshold.
3. Extend `tests.py` with a test that tenant A's widget key cannot create tickets visible to tenant B.
4. Replace the lexicon sentiment with counts weighted by the priority phrase; re-run and compare on the corpus.
5. Implement token expiry in `sessions` (add a `created` column and a TTL check in `tenant_for_token()`); write the test that proves an expired token gets a 401.
6. Add a `GET /api/tickets?status=open` filter end-to-end (handler, function, dashboard, test).

---

# Chapter 12 — How to Run

```
cd resolvedesk-ai
python3 train_ml.py    # once: trains the triage classifier -> results.json + triage_model.npz
python3 app.py         # seeds demo tenants, serves http://localhost:8090
python3 tests.py       # 13 assert-based tests (temp DB, no server needed)
python3 smoke_test.py  # full user journey over real HTTP (starts its own server, port 8199)
```

Requirements: Python 3.12, NumPy. No other dependencies, no API keys, no network access. Demo logins: `demo@urbankart.in` / `demo1234` (UrbanKart, 10 KB articles) and `demo@fitlife.in` / `demo1234` (FitLife Gym, 5 KB articles). Pages: `/` landing, `/dashboard` owner dashboard, `/widget-demo` widget on a fictional store. Environment overrides: `PORT`, `RESOLVEDESK_DB`.

# Chapter 13 — References


1. Salton, G., Wong, A., Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11).
2. Jones, K. S. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1). (IDF.)
3. Robertson, S., Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4). (Context for the retrieval upgrade path.)
4. Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*. (The architecture family the chat engine belongs to.)
5. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*, §4.3.4. (Softmax regression and cross-entropy gradients.)
6. Geifman, Y., El-Yaniv, R. (2017). Selective classification for deep neural networks. *NeurIPS 2017*. (The reject-option theory behind confidence gating.)
7. Abadi, M., et al. — and standard references for PBKDF2: RFC 8018, *PKCS #5: Password-Based Cryptography Specification v2.1* (2017).
8. OWASP. *Password Storage Cheat Sheet* and *Session Management Cheat Sheet*. (Production hardening baseline cited in Chapter 9.)
9. ResolveDesk AI project repository — `README.md`, `BUSINESS.md`, `results.json` (primary sources for all product claims and metrics in this report).

# Appendix A — Full `results.json`


```
{
  "dataset_size": 600,
  "train_size": 480,
  "training_seconds": 14.12,
  "intent_accuracy": 1.0,
  "intent_macro_f1": 1.0,
  "priority_accuracy": 0.9833333333333333,
  "priority_macro_f1": 0.9833147942157954,
  "n_test": 120,
  "model_file": "triage_model.npz"
}
```

# Appendix B — File Listing

```
resolvedesk-ai/
├── README.md            product overview, API reference, limitations
├── BUSINESS.md          business & freelance service pack
├── app.py               server: tenancy, auth, KB, retrieval, chat gate, tickets, analytics,
seeding, HTTP
├── ml.py                triage: data generation, TF-IDF, SoftmaxLR, sentiment, TriageModel
├── train_ml.py          stratified split, training, results.json + triage_model.npz
├── tests.py             13 assert-based tests (temp DB)
├── smoke_test.py        10-step real-HTTP user journey
├── results.json         measured triage metrics (Appendix A)
├── triage_model.npz     trained model artefact (25 KB)
└── web/
    ├── index.html       landing + pricing
    ├── dashboard.html   owner dashboard
    └── widget.html      embeddable widget demo (fictional UrbanKart store)
```

# Appendix C — Complete API Reference

| Method & path | Auth | Purpose |
|---|---|---|
| POST `/api/register` | — | Create tenant  returns id, name, widget key (201; 400 on duplicate email) |
| POST `/api/login` | — | Owner login  session token (401 on bad credentials) |
| GET `/api/me` | Bearer | Tenant info + widget key |
| GET `/api/kb` | Bearer | List the tenant's KB articles |
| POST `/api/kb` | Bearer | Add a KB article (201) |
| DELETE `/api/kb/<id>` | Bearer | Delete own article (tenant-scoped) |
| POST `/api/chat` | widget key in body | Customer chat  cited answer or ticket + triage (404 on bad key) |
| GET `/api/tickets` | Bearer | Tenant's tickets, urgent-first |
| POST `/api/tickets/<id>/status` | Bearer | Set status: open / in-progress / resolved (owner only) |
| GET `/api/analytics` | Bearer | Conversations, deflection rate, tickets, avg confidence, intent breakdown |
| GET `/api/demo-tenants` | — | Seeded tenants + widget keys (demo builds only) |

*End of report — ResolveDesk AI, Ashutosh Sahoo, November 2026. Every metric in this document is taken from the project's `results.json` or from the printed output of its own test suites; every price and cost figure is taken from `BUSINESS.md` and labelled as an estimate where the source labels it so.*